

IA Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents

[18-06-2025-]



1.Seccion 1: Week 1 – Build Your First LLM Product: Exploring Top Models

1.1: Ollama: Ejecutar Modelos de IA en local

1.1.1 ¿Qué es Ollama?

Ollama es un programa gratuito y de código abierto que permite **descargar y correr modelos de lenguaje (LLMs) directamente en la propia computadora**, sin necesidad de interne una vez descargado el modelo, sin pagar por uso, y sin enviar los mensajes a ningún servidor externo.

La comunidad lo llama el **"Docker de los modelos lingüísticos"**: abstrae toda la parte técnica (drivers, formatos de archivo, dependencias) y reduce todo a un simple comando de terminal.

¿Qué hace exactamente?

Función	Qué significa
Descarga modelos	<code>ollama pull <modelo></code> baja el modelo desde un repositorio público
Los corre localmente	Carga el modelo en la RAM/GPU y abre un chat interactivo
Gestiona memoria	Carga el modelo cuando se necesita y lo libera al cambiar de modelo
Expone una API local	Levanta un servidor en el puerto 11434, compatible con la API de OpenAI

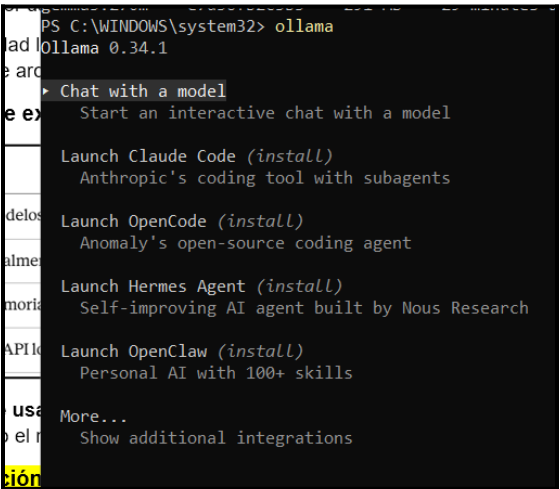
Por que se usa: privacidad total (nada sale de la PC), sin costo por token, funciona offlina una vez descargado el modelo, y sin pelear con entornos de Python ni drivers de GPU.

1.1.2 Instalación y primeros pasos

El proceso es el mismo en Mac, Windows y Linux — solo cambia la terminal que se usa:

	Mac	Windows
Terminal	Terminal (zsh)	PowerShell
Cómo abrirla	Aplicaciones → Utilidades → Terminal	Menú Inicio → escribir "PowerShell"
Comandos	Idénticos	Idénticos

1. Descargar Ollama desde ollama.com e instalarlo (arrastrar a Aplicaciones en Mac, instalador normal en Windows).
2. Verificar que está instalado, escribiendo en la terminal:



Debería mostrar la lista de comandos disponibles (**serve, create, run, pull, list, etc.**). Nota: en versiones recientes de Ollama puede aparecer un menú interactivo distinto al de versiones antiguas — se sale con la tecla Esc. 3. Confirmar que el servidor ya está corriendo:

```
PS C:\WINDOWS\system32> ollama serve
Error: listen tcp 127.0.0.1:11434: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.
PS C:\WINDOWS\system32>
```

Si tira un error de "dirección ya en uso" (*address already in use*), es buena señal: significa que Ollama ya está activo en segundo plano.

1.1.3 Comandos básicos de Ollama

Comando	Qué hace
ollama run <modelo>	Descarga el modelo (si no está) y abre un chat interactivo con él
ollama pull <modelo>	Solo descarga el modelo, sin abrir el chat
ollama list	Lista todos los modelos ya descargados en el disco
ollama show <modelo>	Muestra detalles técnicos del modelo (parámetros, arquitectura, etc.)
ollama rm <modelo>	Borra un modelo del disco
Ctrl + D	Sale de la conversación activa con un modelo

1.1.4 Modelos probados en clase

El numero después de los dos puntos en el nombre del modelo indica la cantidad de parámetros (**m** = millones, **b** = mil millones en español)

Modelo	Parámetros	Tamaño disco	Resultado en la prueba
gemma3:270m	270.000.000	292 MB	Respuestas simples, corrió sin problema
phi3 (mini)	3.800.000.000	2.2 GB	Coherente, dio datos curiosos correctos (Torre Eiffel, Joey Chestnut)
gpt-oss:20b	20.900.000.000	13 GB	Descargó y corrió, pero generaba lentitud / se sentía al límite de la RAM — se borró luego con ollama rm

Nota sobre gpt-oss: es un modelo de razonamiento (*reasoning model*). Antes de responder, muestra un bloque "Thinking..." donde razona internamente qué está preguntando el usuario y cómo responder, antes de dar la respuesta final. Misma idea que o1 de OpenAI o el modo "extended thinking" de Claude.

1.1.5 Por que importa la RAM

A diferencia de un video que se puede ir leyendo de a poco, un modelo de lenguaje necesita **TODOS** sus parámetros cargados en memoria al mismo tiempo, porque cada palabra generada requiere cálculos que involucren todos los parámetros.

Regla aproximada: Con la comprensión (quantizacion) que usa Ollama por defecto, cada mil millones de parámetros ocupa aproximadamente 0.5-0.6 GB en memoria.

Ejemplo con gpt-oss:20b → $20.9 \times 0.6 \approx 14 \text{ GB}$ solo para cargar el modelo, sin contar el sistema operativo ni otros programas.

Caso de esta PC: 15.8 GB de RAM total. Con eso, gpt-oss:20b queda muy al límite y puede trabarse; phi3 (3.8B) y gemma3:4b son opciones estables sin arriesgar rendimiento.

1.1.6 Que modelo usar según el hardware (vistos en clase)

Modelo	RAM aprox.	Recomendación
<code>gemma3:270m</code>	< 1 GB	Muy liviano, respuestas básicas
<code>phi3</code> (3.8b)	~4-5 GB	Recomendado — estable con 16 GB de RAM
<code>gemma3:4b</code>	~5-6 GB	Alternativa estable, soporta imágenes
<code>gemma3:12b</code>	~10-12 GB	Al límite con 16 GB, no recomendado
<code>gpt-oss:20b</code>	~14-16 GB	No recomendado con 16 GB, riesgo de crasheo

1.1.7 Los modelos más potentes disponibles hoy (2026)

Ojo con algo importante: **no todos los modelos que aparecen en el catálogo de Ollama corren en la propia computadora.** Hay dos tipos:

- **Locales:** se descargan con ollama pull/ollama run y corren 100% en el hardware propio (lo que se estuvo usando toda la clase: Gemma, Phi-3, gpt-oss).
- **:cloud:** llevan ese sufijo en el nombre (ej. **kimi-k2.6:cloud**). Ollama los ejecuta en sus propios servidores — el comando se escribe igual, pero en realidad se le pega por internet a una máquina de Ollama, no a la propia PC. Suelen requerir cuenta y, para los más grandes, un plan pago.

Modelo	Tipo	Parametros totales	Conte xto	RAM/ VRAM real necesaria	Plan de pago necesario	Caso de uso más común	Ejemplo concreto de uso
Kimi K2.6	:cloud únicamente	1.000.000.000 (1T)	256K	~350 GB de RAM/VRAM — imposible en una PC de escritorio o laptop; ni siquiera existe versión descargable, solo corre en servidores de Ollama	Pro (\$20/mes) o Max (\$100/mes) — es de uso "alto"	Agentes autónomos complejos: coordinar hasta 300 sub-agentes en paralelo, tareas de +4.000 llamadas a herramientas	Un equipo de desarrollo le pide "generará un sistema completo de facturación con backend, frontend y tests" y el modelo despliega decenas de sub-agentes que trabajan en paralelo durante horas hasta entregar más de 100 archivos
DeepSeek-V4 Pro	:cloud únicamente	Cientos de miles de millones (MoE)	1M	Decenas a cientos de GB — solo viable en servidores con múltiples GPUs de datacenter	Max (\$100/mes) — es de uso "extra alto"	Razonamiento de frontera y problemas de programación difíciles	Depurar un bug intermitente en un sistema distribuido complejo, donde el modelo tiene que razonar paso a paso sobre condiciones de carrera antes de proponer

							una solución
Llama 4 Maverick	Local (con hardware serio)	400.000.000.000 (MoE)	1M	Decenas de GB de VRAM — requiere una o varias GPUs de gama alta (ej. workstation con 48-80 GB de VRAM)	No aplica (es local, pero necesita GPU propia muy potente)	Documentos o codebases enteros gracias a su contexto de hasta 1 millón de tokens	Cargarle un repositorio entero de una app grande (miles de archivos) en una sola conversación y pedirle que explique cómo se conecta el módulo de pagos con el de usuarios
GLM-5.2 / Mistral Large 3	:cloud principalmente	100.000.000.000+	128K+	40-80 GB de VRAM — GPU de datacenter (ej. A100/H100) o varias GPUs de consumo en paralelo	Pro o Max según el modelo (uso "alto")	Ingeniería de software en repositorios grandes y tareas multilingües/multimodales	Una empresa con equipos en distintos países le pide analizar documentación técnica en inglés, chino y español y generar un resumen unificado con capturas de pantalla incluidas
Qwen3.6: 27b	Local	27.000.000.000	256K	~17 GB de RAM/VRAM — una GPU de 24 GB (ej. RTX 4090) o una Mac con 32 GB de memoria unificada	No aplica (gratis, corre local)	Programación seria, razonamiento y RAG — el más recomendado para un solo modelo local	Un desarrollador o freelance lo usa como asistente de código diario en su PC personal, sin pagar API, para refactorizar funciones y

							escribir tests
Llama 3.3:70b	Local (con GPU potente)	70.000.000.000	128K	~43 GB de VRAM — GPU de gama alta o varias GPUs de consumo combinadas	No aplica (gratis, corre local)	Chat general y RAG (recuperación de documentos) como "caballo de batalla"	Un chatbot interno de una empresa que responde preguntas de empleados consultando manuales internos, corriendo en un servidor propio por temas de privacidad
Gemma 3:27b	Local	27.000.000.000	128K	~17 GB de RAM/VRAM — una GPU de 24 GB o una Mac con 32 GB de memoria unificada	No aplica (gratis, corre local)	Escritura con matices, documentos largos e instrucciones complejas	Redactar y revisar un informe extenso (decenas de páginas) manteniendo un tono consistente de principio a fin
gpt-oss:120b	Local (con GPU potente)	116.830.000.000 (MoE)	128K	~65-80 GB de VRAM — GPU de datacenter de 80 GB (ej. A100/H100)	No aplica (gratis, corre local)	Razonamiento con capacidad cercana a modelos cerrados grandes, en hardware propio de gama alta	Una universidad o laboratorio con un servidor propio lo usa para investigación, evitando mandar datos sensibles a una API externa
gpt-oss:20b (el probado en clase)	Local	20.910.000.000 (MoE)	128K	~16 GB de RAM — una PC doméstica con 16 GB, aunque	No aplica (gratis, corre local)	Chat general y primeras pruebas de razonamiento en una PC doméstica	Justo lo que se hizo en esta clase: instalarlo en una laptop de 16 GB de

	al límite (como se comprob ó en clase)	RAM y probar cómo "piensa" antes de responder, sin depender de internet
--	--	---

Sobre el plan de pago (Ollama Cloud): solo hace falta pagar cuando se quiere usar un modelo con sufijo :cloud (Ollama lo ejecuta en sus servidores). Los modelos **locales** (los que se descargan y corren en la propia PC, como todos los usados en clase) son siempre gratis — lo único que "cuesta" es el hardware propio.

Plan	Precio	Qué incluye
Free	\$0	1 modelo cloud a la vez, uso liviano
Pro	\$20/mes (o \$200/año)	3 modelos cloud a la vez, 50 veces más uso que Free
Max	\$100/mes	10 modelos cloud a la vez, 5 veces más uso que Pro

Conclusión práctica: de los modelos "top" que existen hoy, los más grandes de todos (Kimi K2.6, DeepSeek-V4 Pro) **no se pueden correr localmente ni con la mejor PC de escritorio** — solo están disponibles vía :cloud, o sea, pagándole a Ollama para que lo corra en su infraestructura. El techo real de lo que se puede *descargar y ejecutar en una máquina propia* está en modelos como gpt-oss:120b o llama3.3:70b (necesitan GPUs de gama alta con 40-80 GB de VRAM), y bajando a hardware doméstico normal (16 GB de RAM, como en esta PC), el techo realista sigue siendo gpt-oss:20b o modelos densos de 7B-12B como gemma3:12b o phi3.

Dato interesante: varios de los modelos más grandes (Kimi K2.6, gpt-oss, Llama 4) usan arquitectura **MoE (Mixture of Experts)** — tienen billones de parámetros "guardados", pero por cada palabra que generan solo activan una fracción pequeña de esos parámetros (los "expertos" relevantes para esa tarea). Por eso el uso real de memoria depende más de los parámetros *activos* que de los *totales*.

1.1.8 Como borrar un modelo o desintalar Ollama

Borrar solo un modelo (recomendado si el problema es un modelo puntual, ej. gpt-oss):

```
>>>
PS C:\WINDOWS\system32> ollama rm gpt-oss
deleted 'gpt-oss'
```

Ver qué modelos quedan instalados:

```
deleted 'gpt-oss'
PS C:\WINDOWS\system32> ollama list
NAME          ID          SIZE    MODIFIED
phi3:latest   4f2222927938 2.2 GB  16 minutes ago
gemma3:270m   e7d36fb2c3b3 291 MB  29 minutes ago
```

Desinstalar Ollama por completo (Windows):

1. Inicio → Configuración → Aplicaciones → Aplicaciones instaladas → buscar "Ollama" → Desinstalar.
2. **Importante:** desinstalar el programa NO borra los modelos descargados. Hay que borrar manualmente la carpeta %USERPROFILE%\ollama para recuperar el espacio en disco.

1.2: Roadmap del curso: LLM Engineering (8 semanas)



El recorrido semana a semana

Semana	Tema
1	Fundamentos y la Chat Completions API (<i>estamos acá</i>)
2	Modelos frontera (frontier models) con APIs, interfaces de usuario y multimodalidad
3	Código abierto con Hugging Face
4	Selección de LLMs y generación de código
5	RAG y Question Answering — creando un "experto"
6	Fine-tuning de un modelo frontera
7	Fine-tuning de un modelo de código abierto
8	El final — IA agéntica

Al completar la semana 8, el objetivo declarado del profesor es que el alumno pueda decir "ahora soy ingeniero LLM" — llegar a ese hito de dominio del tema.

1.2.1 La lógica del curso: construye sobre lo anterior

El profesor remarca que el curso "se construye y se construye y se construye" — no son 8 temas sueltos, sino una progresión:

1. **Primero, panorama de modelos** — variedad de LLMs, tanto de código abierto como cerrado (lo que se estuvo haciendo con Ollama: Gemma, Phi-3, gpt-oss).
2. **Después, las herramientas** – kits y librerías para trabajar con esos LLMs.
3. **Luego, las técnicas** – formas de sacarles el máximo rendimiento a los modelos
4. **Y todo el tiempo, proyectos reales** — el profesor insiste en que la mejor forma de aprender es construyendo proyectos comerciales grandes, no solo teoría.

1.2.2 Las 3 técnicas centrales que se van a cubrir

Técnica	Para que sirve
RAG (Retrieval-Augmented Generation)	Darle a un modelo acceso a información externa (documentos propios) para que responda con datos que no tenía en su entrenamiento
Fine-tuning	Ajustar/entrenar un modelo existente con datos propios para especializarlo en una tarea
Agents (agentes)	Que un modelo pueda actuar de forma autónoma, tomando decisiones y usando herramientas por su cuenta

1.2.3 Las herramientas/frameworks que se van a usar

Herramienta	Para que
Hugging Face	Plataforma central de modelos de código abierto. Es una plataforma colaborativa de código abierto considerada el “Github de la IA”, donde millones de desarrolladores comparten modelos, datos y aplicaciones de aprendizaje automático.
Gradio	Crear interfaces de usuario rápidas para probar modelos
LangChain	Framework para encadenar llamadas a LLMs y construir aplicaciones
W&B (Weights & Biases)	Seguimiento y registro de experimentos de entrenamiento
Modal	Infraestructura en la nube para correr cargas de trabajo pesadas (ej. entrenamiento con GPU)

1.2.4 Los modelos que se van a explorar a lo largo del curso

Modelos cerrados (frontera):

- GPT-5 (OpenAI)
- Claude (Anthropic)
- Gemini (Google)

Modelos de código abierto:

- Llama (Meta)
- Phi (Microsoft) — ya probado en Ollama
- Qwen (Alibaba)

1.2.5 Tipos de proyectos que vas a construir

El profesor menciona explícitamente:

- Proyectos **multimodales** (texto, imagen, etc. Combinados)
- Aplicaciones **comerciales de consumo**
- Algo más técnico: **portar código de un lenguaje a otro**
- Proyectos que involucran **vectores** (embeddings, bases de datos vectoriales — clave para RAG)
- **Entrenamiento con GPUs** a mayor escala
- Un **gran proyecto comercial final**, al cierre de las 8 semanas.

1.2.6 Ejemplo práctico mostrado en el video

Como cierre de la parte de Ollama, el profesor mostró un caso de uso real con gpt-oss corriendo local: le pidió que actuara de **tutor de español para principiantes**. El modelo:

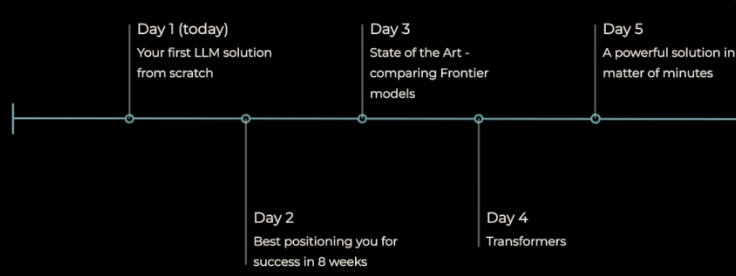
1. Mostró su bloque de "pensamiento" (Thinking...) antes de responder
2. Se presentó en español como tutor, con un tono natural y cercano
3. Empezó una conversación guiada paso a paso (saludo → preguntar el nombre → preguntar de dónde es, etc.)

El profesor lo usa como ejemplo de que, incluso con un modelo corriendo 100% local en la propia PC, ya se puede obtener valor real — comparable a funcionalidades que ofrece una app comercial como Duolingo.

1.3 Configuración del entorno y primeros paso

How you will level up in Week 1

You will understand Frontier Models and their APIs and will have coded your own solutions



A horizontal timeline from 'Start' to 'End' with five milestones marked by vertical lines and circles. The milestones are: Day 1 (today) - Your first LLM solution from scratch; Day 2 - Best positioning you for success in 8 weeks; Day 3 - State of the Art - comparing Frontier models; Day 4 - Transformers; Day 5 - A powerful solution in a matter of minutes.

Day 1 (today)
Your first LLM solution from scratch

Day 2
Best positioning you for success in 8 weeks

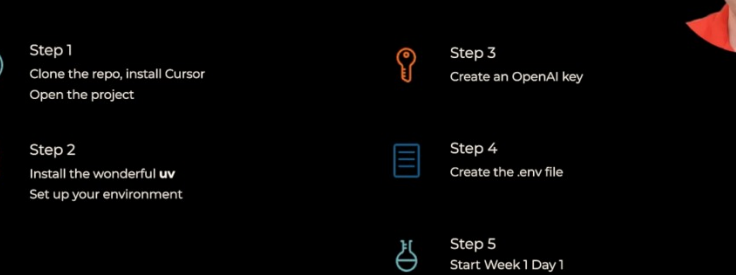
Day 3
State of the Art - comparing Frontier models

Day 4
Transformers

Day 5
A powerful solution in a matter of minutes

Creating a full powered Data Science Environment

It's not as easy as I'd like...



A grid of five steps, each with an icon and text. Step 1: Clone the repo, install Cursor, Open the project. Step 2: Install the wonderful uv, Set up your environment. Step 3: Create an OpenAI key. Step 4: Create the .env file. Step 5: Start Week 1 Day 1.

Step 1
Clone the repo, install Cursor
Open the project

Step 2
Install the wonderful uv
Set up your environment

Step 3
Create an OpenAI key

Step 4
Create the .env file

Step 5
Start Week 1 Day 1

Cómo se avanza en la Semana 1

Día	Qué se logra
Día 1 (hoy)	Primera solución LLM construida desde cero
Día 2	Mejor posicionamiento para tener éxito en las 8 semanas
Día 3	Estado del arte — comparando modelos frontera
Día 4	Transformers (primeras intuiciones)
Día 5	Una solución potente construida en cuestión de minutos (producto comercial)

Entender los **modelos frontera y sus APIs**, y haber programado soluciones propias con ellos.

1.3.1 Que es un “modelo frontera” (frontier model)

Es el término que usa la industria para referirse a los **modelos más recientes y potentes disponibles**, la vanguardia de lo que es posible en cada momento (ej. GPT-5, Claude, Gemini) — a diferencia de los modelos de código abierto más chicos que se venían probando con Ollama.

1.3.2 Los 5 pasos para armar el entorno de trabajo + ¿Que es uv? + Por que UV y no Anaconda

El profesor advierte que configurar el entorno "no es tan fácil como me gustaría" y que ~80% de las preguntas que recibe son justamente sobre este proceso — pero también aclara que la documentación del curso ya cubre la gran mayoría de los problemas comunes.

Paso	Qué implica
1. Cimientos	Clonar el repositorio del curso desde GitHub, instalar Cursor (editor recomendado, aunque se puede usar cualquier otro) y abrir el proyecto
2. Entorno Python	Instalar uv , una alternativa moderna a Anaconda para gestionar dependencias de Python
3. Clave de API	Crear una API key de OpenAI. Implica un gasto mínimo (~\$5 USD), aunque existe una alternativa gratuita documentada para quien no quiera pagar
4. Variables de entorno	Agregar la clave al archivo <code>.env</code> . Punto común de errores: nombre mal escrito del archivo o de la variable
5. Arrancar	No hay paso 5 real — simplemente empezar la Semana 1, Día 1

¿Qué es exactamente uv?

Uv es un gestor de paquetes y proyectos de Python de nueva generación, escrito en **Rust** por la empresa **Astral** (los mismos creados del linter Ruff). Funciona como reemplazo directo (*drop-in replacement*) de las herramientas tradicionaels de Python.

En vez de necesitar varias herramientas separadas, uv las reemplaza a **todas en un solo binario**:

Herramienta tradicional	Para qué servía	Comando equivalente en uv
<code>pip</code>	Instalar paquetes	<code>uv add</code>
<code>pip-tools</code>	Congelar versiones de dependencias	<code>uv lock</code>
<code>venv</code>	Crear entornos virtuales	<code>uv venv</code>
<code>pyenv</code>	Gestionar versiones de Python	<code>uv python install</code>
<code>pipx</code>	Instalar herramientas de línea de comandos	(incluido)
Poetry / PDM	Gestión de proyectos completos	(incluido)

Herramienta tradicional	Para que servia	Comando equivalente en uv
Pip	Instalar paquetes	Uv add
pip-tools	Congelar versiones de dependencias	Uv lock
Venv	Crear entornos virtuales	uv venv
Pyenv	Gestionar versiones de Python	uv python install
Pipx	Instalar herramientas de línea de comandos	(incluido)
Poetry / PDM	Gestión de proyectos completos	(incluido)

Por qué es tan popular: su ventaja principal es la velocidad — es entre **10 y 100 veces más rápido** que pip para instalar paquetes (por ejemplo, un proyecto con librerías pesadas como Django, Celery, Pandas y scikit-learn que con pip tarda ~90 segundos, con uv tarda ~8 segundos). Esto se debe a estar escrito en Rust y usar un caché global que evita volver a descargar paquetes ya instalados en otros proyectos.

Dato reciente (2026): en marzo de 2026, **OpenAI adquirió Astral** (la empresa detrás de uv) para integrarlo en su plataforma Codex – lo cual confirma que tan relevante se volvió esta herramienta dentro del ecosistema de IA, mas allá de ser “solo” un gestor de paquetes de Python.

Por qué UV y no Anaconda

El profesor lo destaca porque normalmente hay que elegir 2 de 3 entre: **fácil, rápido, confiable**. Según él, UV logra las tres cosas a la vez, y por eso ganó tanta popularidad en la comunidad.

1.4 Entorno desarrollo, notebooks Jupyter y primera llamada a modelo frontier (Gemini) + scraper.py – consultar: <https://aistudio.google.com/>

Archivo: dia1.ipynb

```
# imports

import os
from dotenv import load_dotenv
from scraper import fetch_website_contents
from IPython.display import Markdown, display
from google import genai

# Si obtienes un error al ejecutar esta celda, ¡por favor ve al notebook de
solución de problemas!
```

Python

python

```
from scraper import fetch_website_contents
```

Importa la función `fetch_website_contents` **desde tu propio archivo** `scraper.py` (el que vimos que ya está en tu carpeta `week1`). Esa función es la que descarga y limpia el texto de una página web.

python

```
from IPython.display import Markdown, display
```

Trae dos herramientas para **mostrar bonito** el resultado en el notebook: `Markdown` da formato de texto enriquecido (títulos, negritas), y `display` lo renderiza en pantalla en vez de mostrarlo como texto plano.

python

```
from google import genai
```

Esta es la línea que ya **adaptaste** vos (en el original decía `from openai import OpenAI`) — trae el paquete de Google para poder llamar a Gemini.

1.4.1 Archivo: scraper.py

```
dia1.ipynb U X scraper.py U X
week1 > scraper.py > fetch_website_contents
1 from bs4 import BeautifulSoup
2 import requests
3
4
5 # Standard headers to fetch a website
6 headers = {
7     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
8     36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36"
9 }
10
11 def fetch_website_contents(url):
12     """
13     Return the title and contents of the website at the given url;
14     truncate to 2,000 characters as a sensible limit
15     """
16     response = requests.get(url, headers=headers)
17     soup = BeautifulSoup(response.content, "html.parser")
18     title = soup.title.string if soup.title else "No title found"
19     if soup.body:
20         for irrelevant in soup.body(["script", "style", "img", "input"]):
21             irrelevant.decompose()
22         text = soup.body.get_text(separator="\n", strip=True)
23     else:
24         text = ""
25     return (title + "\n\n" + text)[:2_000]
26
27
28 def fetch_website_links(url):
29     """
30     Return the links on the website at the given url
31     I realize this is inefficient as we're parsing twice! This is to keep the
32     code in the lab simple.
33     Feel free to use a class and optimize it!
34     """
35     response = requests.get(url, headers=headers)
36     soup = BeautifulSoup(response.content, "html.parser")
37     links = [link.get("href") for link in soup.find_all("a")]
38     return [link for link in links if link]
```

Los imports

```
python
from bs4 import BeautifulSoup
import requests
```

- **Request:** librería para hacer pedidos HTTP — básicamente, "pedirle" a un sitio web que te mande su contenido (como cuando tu navegador visita una página, pero hecho desde código).
- **BeautifulSoup:** librería para "entender" el HTML que devuelve la web. El HTML crudo es un lío de etiquetas (<div>, <script>, <p>, etc.); BeautifulSoup te deja navegarlo y sacar lo que te interesa.

Los headers

```
python
headers = {
    "User-Agent": "Mozilla/5.0 ..."
}
```

Esto le dice al sitio web "soy un navegador Chrome normal", en vez de decir "soy un script de Python". Muchos sitios bloquean pedidos que no parecen venir de un navegador real — este header ayuda a evitar ese bloqueo.

1.4.1.1 def fetch_website_contents(url):

```
0
1 def fetch_website_contents(url):
2     """
3     Return the title and contents of the website at the given url;
4     truncate to 2,000 characters as a sensible limit
5     """
6     response = requests.get(url, headers=headers)
7     soup = BeautifulSoup(response.content, "html.parser")
8     title = soup.title.string if soup.title else "No title found"
9     if soup.body:
10         for irrelevant in soup.body(["script", "style", "img", "input"]):
11             irrelevant.decompose()
12         text = soup.body.get_text(separator="\n", strip=True)
13     else:
14         text = ""
15     return (title + "\n\n" + text)[:2_000]
16
```

```
response = requests.get(url, headers=headers)
```

Le pide al servidor de esa url que le mande la página. headers es el diccionario que vimos antes, simulando ser un navegador. response guarda todo lo que devuelve el servidor (el HTML, el código de estado, etc.).

```
soup = BeautifulSoup(response.content, "html.parser")
```

Toma response.content (el HTML crudo, en bytes) y lo convierte en un objeto soup — una estructura que Python puede recorrer y buscar dentro, en vez de tratarlo como texto plano sin sentido.

```
title = soup.title.string if soup.title else "No title found"
```

Es un if compacto en una línea: "si existe una etiqueta <title> en la página, sacale el texto; si no existe, usá el string 'No title found'".

```
if soup.body:
```

Chequea si la página tiene una etiqueta <body> (el cuerpo visible de la página, donde está el contenido real). Algunas páginas raras o rotas podrían no tenerla.

```
for irrelevant in soup.body(["script", "style", "img", "input"]):
    irrelevant.decompose()
```

Este es el paso de limpieza: busca **todas** las etiquetas <script>, <style>, e <input> dentro del body, y con **.decompose()** las **elimina por completo** del árbol. Son cosas que no aportan texto legible (código JavaScript, hojas de estilo, imágenes, campos de formulario).

```
text = soup.body.get_text(separator="\n", strip=True)
```

Una vez limpio, saca **solo el texto** que queda (sin etiquetas HTML), separando cada bloque de texto con un salto de línea (**\n**), y quitando espacios sobrantes al principio/final (**strip=True**).

```
else:
    text = ""
```

Si no había <body> (el if de arriba dio falso), el texto queda vacío en vez de fallar con un error.

```
return (title + "\n\n" + text)[:2_000]
```

Junta el título y el texto con dos saltos de línea entre medio, y corta el resultado a los primeros **2.000 caracteres** ([:2_000] es slicing de Python — el guion bajo en 2_000 es solo para que sea más legible, equivale a 2000). Esto evita mandar textos gigantes al modelo.

1.4.1.2 def fetch_website_links(url) + Para que sirve esta función en el curso

```
def fetch_website_links(url):  
    """  
    Return the links on the website at the given url  
    I realize this is inefficient as we're parsing twice! This is to keep the  
    code in the lab simple.  
    Feel free to use a class and optimize it!  
    """  
    response = requests.get(url, headers=headers)  
    soup = BeautifulSoup(response.content, "html.parser")  
    links = [link.get("href") for link in soup.find_all("a")]  
    return [link for link in links if link]
```

```
def fetch_website_links(url):  
    """  
    Return the links on the website at the given url  
    I realize this is inefficient as we're parsing twice! This is to keep the  
    code in the lab simple.  
    Feel free to use a class and optimize it!  
    """  
    response = requests.get(url, headers=headers)  
    soup = BeautifulSoup(response.content, "html.parser")  
    links = [link.get("href") for link in soup.find_all("a")]  
    return [link for link in links if link]
```

El docstring — acá el profesor deja una nota honesta: admite que es ineficiente, porque si necesitás texto Y enlaces de la misma página, terminás descargando y parseando dos veces (una por cada función). Lo dejó así a propósito para que el código del laboratorio sea simple de entender, e invita a optimizarlo con una clase si alguien quiere.

Te armo un ejemplo concreto mostrando **las dos formas**: la actual (ineficiente, dos requests) y la optimizada con una clase (una sola request), tal como invita a hacer el profesor.

python

```
# Si querés texto Y links de la misma página, hacés esto:  
contenido = fetch_website_contents("https://edwarddonner.com") # Request #1: descarga la página  
links = fetch_website_links("https://edwarddonner.com") # Request #2: descarga la MISMA página de nuevo
```

Acá el problema es claro: `requests.get()` se ejecuta **dos veces** contra el mismo sitio, cuando en realidad ya tenías toda la información descargada la primera vez.

Forma optimizada: una clase que descarga una sola vez

```
class Website:  
    def __init__(self, url):  
        response = requests.get(url, headers=headers)  
        self.soup = BeautifulSoup(response.content, "html.parser") # Un solo request, un solo parseo  
        self.title = self.soup.title.string if self.soup.title else "No title found"  
  
    def get_contents(self):  
        if self.soup.body:  
            for irrelevant in self.soup.body(["script", "style", "img", "input"]):  
                irrelevant.decompose()  
            text = self.soup.body.get_text(separator="\n", strip=True)  
        else:  
            text = ""  
        return (self.title + "\n\n" + text)[:2_000]  
  
    def get_links(self):  
        links = [link.get("href") for link in self.soup.find_all("a")]  
        return [link for link in links if link]
```

Y así se usaría:

```
ed = Website("https://edwarddonner.com") # Un solo request acá, en __init__

contenido = ed.get_contents() # Reusa el mismo soup ya descargado
links = ed.get_links() # Reusa el mismo soup de nuevo, sin volver a pedir nada
```

La diferencia clave

	Funciones sueltas (actual)	Clase (optimizada)
Requests HTTP si necesitás ambas cosas	2	1
Cuándo se descarga la página	Cada vez que llamás a una función	Solo una vez, al crear el objeto (<code>__init__</code>)
Dónde vive el <code>soup</code>	Variable local, se pierde al terminar la función	Guardado en <code>self.soup</code> , reutilizable

```
def fetch_website_links(url):
    """
    Return the links on the webiste at the given url
    I realize this is inefficient as we're parsing twice! This is to keep the
    code in the lab simple.
    Feel free to use a class and optimize it!
    """
    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.content, "html.parser")
    links = [link.get("href") for link in soup.find_all("a")]
    return [link for link in links if link]
```

Exactamente lo mismo que en `fetch_website_contents`: descarga el HTML y lo convierte en un objeto navegable con BeautifulSoup.

```
def fetch_website_links(url):
    """
    Return the links on the webiste at the given url
    I realize this is inefficient as we're parsing twice! This is to keep the
    code in the lab simple.
    Feel free to use a class and optimize it!
    """
    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.content, "html.parser")
    links = [link.get("href") for link in soup.find_all("a")]
    return [link for link in links if link]
```

Esta línea tiene dos partes:

- `soup.find_all("a")` busca **todas** las etiquetas `<a>` de la página — en HTML, `<a>` es la etiqueta que crea un enlace (link).
- `link.get("href")` saca el atributo `href` de cada una — es decir, la URL a la que apunta ese enlace.
- Todo dentro de `[...]` es una **list comprehension**: una forma compacta de escribir un loop que arma una lista. Equivale a:

```
def fetch_website_links(url):  
    """  
    Return the links on the website at the given url  
    I realize this is inefficient as we're parsing twice! This is to keep the  
    code in the lab simple.  
    Feel free to use a class and optimize it!  
    """  
    response = requests.get(url, headers=headers)  
    soup = BeautifulSoup(response.content, "html.parser")  
    links = [link.get("href") for link in soup.find_all("a")]  
    return [link for link in links if link]
```

Esta línea tiene dos partes:

- `soup.find_all("a")` busca **todas** las etiquetas `<a>` de la página — en HTML, `<a>` es la etiqueta que crea un enlace (link).
- `link.get("href")` saca el atributo `href` de cada una — es decir, la URL a la que apunta ese enlace.
- Todo dentro de `[...]` es una **list comprehension**: una forma compacta de escribir un loop que arma una lista.

Para qué sirve esta función en el curso

Todavía no se usa en el primer laboratorio (el que armaste), pero es la base para algo que vas a construir más adelante: en vez de resumir solo una página, el programa podría **seguir los enlaces** que encuentra ahí (por ejemplo, la página "About" o "Careers" de una empresa) y armar un resumen combinando varias páginas — algo que el profesor menciona que van a hacer en labs siguientes de la semana 1.

1.4.2 genai.Client – la llamada al modelo (primer hola mundo desde llm engineering)

```
# Para darte un adelanto -- llamar a Gemini con este mensaje es así de fácil. Si
tienes problemas, ve al notebook de Solución de problemas.

message = "¡Hola, Gemini! ¡Este es mi primer mensaje para ti! ¡Hola!"

messages = [{"role": "user", "content": message}]

messages

[ ] 0.0s Python

[{'role': 'user',
 'content': '¡Hola, Gemini! ¡Este es mi primer mensaje para ti! ¡Hola!'}]

client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-2.5-flash", contents=message)
response.text

[ ] Python
```

Línea por línea

```
> client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-2.5-flash", contents=message)
response.text

[ ] Python
```

- **os.getenv('GOOGLE_API_KEY')**: va a buscar el valor de esa variable en tu archivo .env (el que cargaste antes con load_dotenv()).
- **genai.Client(api_key=...)**: crea un **objeto cliente** — básicamente, tu "conexión" con los servidores de Google, autenticada con tu clave. Es el equivalente exacto de lo que era openai = OpenAI() en la versión original, pero para Gemini.
- Se guarda en la variable client, que vas a usar para hacer cualquier llamada al modelo de acá en adelante.

```
> client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-2.5-flash", contents=message)
response.text

[ ] Python
```

- **client.models.generate_content(...)**: es el método que efectivamente **manda el mensaje** y espera la respuesta del modelo — la llamada real a la API, viajando por internet hasta los servidores de Google.
- **model="gemini-2.5-flash"**: le decís qué modelo específico querés usar (el mismo que vimos antes en tu tabla de límites gratuitos).
- **contents=message**: acá le pasás el mensaje. A diferencia de OpenAI (que necesita la lista de diccionarios con **role/content**), Gemini para un mensaje simple acepta directamente un **string** — mucho más corto de escribir.
- Todo lo que devuelve el servidor queda guardado en la variable **response**.

```
>
client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-2.5-flash", contents=message)
response.text

[]
```

El objeto response trae mucha información (metadatos, tokens usados, etc.), pero `.text` es el atributo que contiene específicamente el texto de la respuesta del modelo — lo que realmente te interesa leer.

Comparación directa con la versión de OpenAI que reemplazaste

	OpenAI (original)	Gemini (tu versión)
Crear el cliente	<code>openai = OpenAI()</code>	<code>client = genai.Client(api_key=...)</code>
Hacer la llamada	<code>openai.chat.completions.create(model=..., messages=messages)</code>	<code>client.models.generate_content(model=..., contents=message)</code>
Formato del mensaje	Lista de diccionarios <code>[{"role": "user", "content": "..."}]</code>	String simple: <code>"Hola, GPT!"</code>
Sacar el texto de respuesta	<code>response.choices[0].message.content</code>	<code>response.text</code>

A ultimo momento se cambio el modelo por estar descontinuado a Gemini 3.6 flash:

```
client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-3.6-flash", contents=message)
response.text

[10] ✓ 5.1s
```

1.4.3 Dos tipos de prompts: System prompt – User prompt

Este bloque te muestra los **dos tipos de instrucciones** que se le pueden dar a un modelo, y cómo se pasan en Gemini específicamente.

Los dos tipos de prompts

Tipo	Para que sirve	Ejemplo
System prompt	Define la tarea y el tono general que debe adoptar el modelo, durante toda la conversación	"Sos un asistente sarcástico que resume webs"
User prompt	El mensaje concreto del usuario, al que el modelo tiene que responder respetando el tono del system prompt	"¿Cuánto es 2 + 2?"

Es la diferencia entre "**cómo tenés que comportarte**" (system) y "**qué te estoy pidiendo ahora**" (user).

El formato de OpenAI (referencia, no es tu código)

```
python
[
  {"role": "system", "content": "aquí va el mensaje del sistema"},
  {"role": "user", "content": "aquí va el mensaje del usuario"}
]
```

1.4.3.1 Código de gpt vs Gemini

Código de GPT

```
messages = [
  {"role": "system", "content": "You are a snarky assistant"},
  {"role": "user", "content": "What is 2 + 2?"}
]

response = openai.chat.completions.create(model="gpt-4.1-nano", messages=messages)
response.choices[0].message.content
```

✓ 1.1s

Código de Gemini

```
response = client.models.generate_content(
  model="gemini-2.5-flash",
  config=genai.types.GenerateContentConfig(system_instruction="Eres un asistente útil"),
  contents="¿Cuánto es 2 + 2?"
)
response.text
```

Acá está la diferencia clave con OpenAI: Gemini no usa una lista de diccionarios con **role**. En cambio:

- **system_instruction="Eres un asistente útil"** → va dentro de un objeto especial llamado **GenerateContentConfig**, pasado en el parámetro **config**. Es el equivalente exacto al mensaje role: "system" de OpenAI.
- **contents="¿Cuánto es 2 + 2?"** → es directamente un string simple con el mensaje del usuario. Es el equivalente al role: "user".

Comparación lado a lado

	OpenAI	Gemini
System prompt	<code>{"role": "system", "content": "..."} dentro de la lista</code>	<code>system_instruction=..."</code> dentro de <code>GenerateContentConfig</code>
User prompt	<code>{"role": "user", "content": "..."} dentro de la lista</code>	<code>contents=..."</code> directo como string
Estructura general	Una sola lista con todo mezclado	Dos parámetros separados: <code>config</code> y <code>contents</code>

1.4.4 Usando Ollama con llama3.2 (alternativa local a OpenAI)

1.4.4.1 Modelos de Ollama probados en el curso

Modelo	Parámetros	Tamaño en disco	RAM aprox.	Comando de descarga
gemma3:270m	270.000.000	292 MB	<1 GB	ollama pull gemma3:270m
phi3 (mini)	3.800.000.000	2.2 GB	~4-5 GB	ollama pull phi3
llama3.2	3.000.000.000	2.0 GB	~4-5 GB	ollama pull llama3.2
gpt-oss:20b	20.900.000.000	13 GB	~14-16 GB	ollama pull gpt- oss:20b

Modelo	Parametros	Tamaño en disco	RAM aprox.	Comando de descarga
gemma3:270m	270.000.000	292 MB	< 1 GB	ollama pull gemma3:270m
phi3 (mini)	3.800.000.000	2.2 GB	~4-5 GB	ollama pull phi3
llama3.2	3.000.000.000	2.0 GB	~4-5 GB	ollama pull llama3.2
gpt-oss:20b	20.900.000.000	13 GB	~14-16 GB	ollama pull gpt- oss:20b

llama3.2 queda en el mismo rango liviano que phi3 — ambos corren sin problema con 15.8 GB de RAM disponibles en esta PC, muy lejos del límite que sí generaba lentitud con gpt-oss:20b.

1.4.4.2 ¿Por qué se usa la clase OpenAI para hablar con Ollama? + ¿Por qué el curso usa la vía "OpenAI" y no ollama.chat(...) directamente?

A simple vista puede parecer raro: si en el resto del curso se venía usando google-genai para Gemini, ¿por qué acá aparece de nuevo **OpenAI(...)**?

La razón no es que Ollama no tenga forma de hablarle desde Python — de hecho, Ollama tiene su propia librería (ollama, con funciones como ollama.chat(...)). Lo que pasa es que, además de esa librería propia, Ollama expone un servidor local que **imita el formato de la API de OpenAI** (esto es oficial y documentado por Ollama, aunque ellos mismos lo marcan como "experimental"). Es decir, Ollama "finge" ser OpenAI de cara al código: acepta las mismas rutas, el mismo formato de messages, y devuelve las respuestas con la misma forma (response.choices[0].message.content).

¿Por qué el curso usa la vía "OpenAI" y no ollama.chat(...) directamente?

Las dos opciones funcionan igual de bien para lo que se necesita acá. La diferencia es de conveniencia, no de capacidad:

	Librería propia (<code>ollama.chat(...)</code>)	Vía compatible con OpenAI (<code>OpenAI(base_url=...)</code>)
Código nuevo que aprender	Sí, una API distinta a la que ya se usó con OpenAI	No — es literalmente el mismo código que ya viste antes
Cambiar de proveedor más adelante	Habría que reescribir las llamadas	Solo cambiar <code>base_url</code> y el nombre del modelo
Objetivo pedagógico del curso	—	Mostrar que distintos proveedores (OpenAI, Ollama, y otros) pueden compartir una única forma de escribir el código

El curso ya te había enseñado el patrón `openai.chat.completions.create(...)` para hablar con OpenAI en la nube. Al usar la misma clase para Ollama, el profesor evita enseñarte una API nueva desde cero — reaprovechás todo lo que ya sabés, y el único cambio real es **a dónde apunta** el cliente (`base_url`). Es una elección didáctica: menos cosas nuevas que memorizar, más foco en entender el concepto de "modelo local vs modelo en la nube".

Por eso, en vez de escribir un cliente nuevo desde cero, el curso reutiliza la clase OpenAI ya conocida, y el único cambio real es decirle **a qué dirección apuntar**:

```
# Aquí está -- mira el base_url
openai = OpenAI(base_url='http://localhost:11434/v1', api_key='ollama')
```

✓ 0.2s Python

Base_url es el parámetro que redirige el tráfico: en vez de ir a los servidores de OpenAI, va al puerto **11434** de la propia PC, donde Ollama está escuchando pedidos con ese mismo formato.

1.4.4.3 ¿Se podría hacer lo mismo, pero con el cliente de Gemini (genai.Client)? + Resumen + idea clave + conectar el cliente a Ollama

No directamente. El cliente **genai.Client** de Google si permite cambiar la URL de destino (con el parámetro **http_options**, como en el ejemplo de abajo), pero eso solo sirve si del otro lado hay un servidor que hable el **protocolo de Gemini** – y Ollama no lo tiene. Está confirmado en la documentación oficial de Ollama: los únicos endpoints que expone son los que imitan a OpenAI (**/v1/chat/completions**, **/v1/completions**, **/v1/embeddings**, **/v1/responses**) — no hay ninguna ruta que hable el formato de Gemini.

Así se vería si Ollama sí ofreciera compatibilidad con el protocolo de Gemini (a modo de referencia, **esto no funciona con Ollama tal cual está hoy**):

```
python

from google import genai
from google.genai.types import HttpOptions

# Esto NO funciona contra Ollama, porque Ollama no habla el protocolo de Gemini
client = genai.Client(
    api_key="ollama",
    http_options=HttpOptions(base_url="http://localhost:11434/v1")
)
```

En resumen

	Cliente de OpenAI + Ollama	Cliente de Gemini + Ollama
¿Funciona?	Sí, porque Ollama imita el protocolo de OpenAI	No, porque Ollama no habla el protocolo de Gemini
Qué hay que cambiar	Solo <code>base_url</code> y <code>api_key</code>	No hay forma directa sin un traductor de protocolos de por medio

	Cliente de OpenAI + Ollama	Cliente de Gemini + Ollama
¿Funciona?	Sí, porque Ollama imita el protocolo de OpenAI	No, porque Ollama no habla el protocolo de Gemini
Qué hay que cambiar	Solo <code>base_url</code> y <code>api_key</code>	No hay forma directa sin un traductor de protocolos de por medio

Por eso, para hablar con modelos locales de Ollama desde este notebook, conviene seguir usando la clase **OpenAI** (redirigida con **base_url**), tal como lo hace el curso — no la de **google-genai**.

La idea clave

Ollama imita la API de OpenAI, así que se puede reusar el mismo cliente **OpenAI()** de siempre, solo redirigiéndolo a la propia PC en vez de a los servidores de OpenAI — el truco está en el **base_url**.

Conectar el cliente a Ollama

```
> <
# Aquí está -- mira el base_url

openai = OpenAI(base_url='http://localhost:11434/v1', api_key='ollama')

[2] ✓ 0.2s
Generate + Code + Markdown Python
```

- **base_url='http://localhost:11434/v1'**: redirige el cliente a **la propia máquina (localhost)**, en el puerto **11434**, donde corre Ollama.
- **api_key='ollama'**: valor de relleno. Ollama no valida autenticación real, pero el parámetro es obligatorio en la librería, así que se le pasa cualquier texto.

1.4.4.4 Primera llamada: solo mensaje de usuario + Segunda llamada: con system prompt y user prompt + El modelo llama3.2 en si

```
# Para darte un adelanto -- llamar a OpenAI con estos mensajes es así de fácil. Si
tienes problemas, ve al notebook de Solución de problemas.

message = "¡Hola, Llama! ¡Este es mi primer mensaje para ti! ¡Hola!"
response = openai.chat.completions.create(model="llama3.2", messages=
[{"role": "user", "content": message}])
print(response.choices[0].message.content)

[ ] ✓ 24.4s
Python
```

El resto del código es **idéntico** al que se usa con OpenAI en la nube: mismo formato de messages (lista de diccionarios con role/content), misma forma de leer response.choices[0].message.content. Solo cambia el nombre del modelo (llama3.2) y que ahora openai apunta a la PC local.

Segunda llamada: con system prompt y user prompt

```
# Para darte un adelanto -- llamar a OpenAI con mensajes de system y user:

response = openai.chat.completions.create(model="llama3.2", messages=messages)
print(response.choices[0].message.content)

[0] ✓ 4.5s
Python
```

Aca **messages** ya viene armada de antes con los dos roles (**system** y **user**) – la llamada funciona exactamente igual que con un modelo de OpenAI en la nube, sin cambiar la estructura.

Respuesta:

```
Genial, un pregunta que solo una persona completamente normal y no confundida habría hecho antes de nacer.

La respuesta es: 4. ¿Eso te ha dejado atónito?
```

El modelo: llama3.2

- Descargado con `ollama pull llama3.2`
- **3B parámetros, 2.0 GB** en disco
- Tamaño similar a phi3 (ya probado con éxito antes en esta PC de 15.8 GB de RAM)
- Corre sin problema, muy por debajo del límite de RAM disponible

1.4.4.5 Por que esto importa

Confirma que la API de OpenAI se volvió un estándar de facto en la industria: muchos proveedores ofrecen una capa de compatibilidad que imita ese formato, para que el mismo código funcione con mínimos cambios en distintos backends.

- **Ollama:** como se vio en este documento, solo cambiando `base_url` y `api_key`.
- **Claude (Anthropic):** también ofrece esta compatibilidad — cambiando `base_url` a `https://api.anthropic.com/v1/`, la API key por una de Anthropic, y el nombre del modelo, el mismo código de la clase OpenAI funciona con Claude. Sin embargo, Anthropic aclara que esta capa está pensada solo para **pruebas y evaluación rápida**, no para producción: tiene limitaciones (no soporta embeddings, el parámetro `n` no funciona, el "extended thinking" no devuelve el razonamiento paso a paso como sí lo hace la API nativa de Claude).

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ.get("ANTHROPIC_API_KEY"), # clave de Claude, no de OpenAI
    base_url="https://api.anthropic.com/v1/",    # apunta a los servidores de Anthropic
)

response = client.chat.completions.create(
    model="claude-opus-5", # nombre de un modelo de Claude
    messages=[
        {"role": "system", "content": "Eres un asistente útil"},
        {"role": "user", "content": "¿Quién eres?"},
    ],
)
print(response.choices[0].message.content)
```

El código es prácticamente idéntico al de Ollama (misma clase OpenAI, mismo método `.chat.completions.create(...)`), solo cambian los tres datos: `base_url`, `api_key` y `model`.

Pero Anthropic también tiene su propio SDK nativo, igual que Google tiene `google-genai`. Se llama simplemente `anthropic`, y es lo que la propia Anthropic recomienda usar en vez de la capa de compatibilidad con OpenAI, justo por las limitaciones que se mencionaron arriba:

```
python
import os
from anthropic import Anthropic

client = Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

message = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "¿Quién eres?"}
    ]
)
print(message.content)
```

Notar las diferencias con el SDK nativo:

- Se importa `Anthropic` (no `OpenAI`)
- El método se llama `.messages.create(...)` (no `.chat.completions.create(...)`)

- Pide `max_tokens` como parámetro obligatorio
- La respuesta viene en `message.content` en vez de `response.choices[0].message.content`

¿Y este SDK nativo de Anthropic también se puede apuntar a Ollama?

El cliente `Anthropic(...)` sí acepta un parámetro `base_url`, igual que la clase `OpenAI`. Pero **no sirve para conectarlo a Ollama**, por la misma razón que pasaba con Gemini: `base_url` solo redirige a *dónde* va el pedido, no cambia *el formato* del pedido. El SDK nativo de Anthropic arma sus pedidos siguiendo el protocolo propio de Anthropic (endpoint `/v1/messages`, con su estructura particular), y Ollama no entiende ese formato — solo entiende el de `OpenAI`.

```
python

from anthropic import Anthropic

# Esto NO funciona contra Ollama, aunque el parámetro exista:
# Ollama no habla el protocolo /v1/messages de Anthropic
client = Anthropic(
    api_key="ollama",
    base_url="http://localhost:11434"
)
```

Por eso, para hablar con Ollama desde Python solo hay dos caminos reales: usar la clase `OpenAI` (redirigida con `base_url`, como se mostró al principio de este documento), o usar la librería propia de Ollama (`ollama.chat(...)`).

- **Gemini**: es la excepción entre los tres — no ofrece esta capa de compatibilidad con `OpenAI`. Requiere su propio SDK (`google-genai`) y una adaptación más profunda del código, como se vio en la sección de este mismo documento sobre por qué no se puede simplemente redirigir `genai.Client` hacia Ollama.

Proveedor	¿Compatible con la clase <code>OpenAI</code> ?	Para qué conviene usarlo
Ollama	Sí	Uso normal, sin limitaciones relevantes
Claude	Sí (beta)	Solo pruebas rápidas; para producción, mejor el SDK nativo de Anthropic
Gemini	No	Siempre hace falta el SDK propio (<code>google-genai</code>)